

Database Schema Breakdown

here is the database schema structure for application.

--

-- ENUMS

--

```
CREATE TYPE project_role AS ENUM ('owner',
'member');
CREATE TYPE invitation_status AS ENUM ('pending',
'accepted', 'declined', 'expired');
CREATE TYPE task_status AS ENUM ('backlog', 'todo',
'in_progress', 'in_review', 'done', 'cancelled');
CREATE TYPE task_priority AS ENUM ('none', 'low',
'medium', 'high', 'urgent');
```

--

-- HELPER TRIGGER: auto-update updated_at

--

```
CREATE OR REPLACE FUNCTION handle_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END; $$;
```

--

-- PROFILES

--

```
CREATE TABLE profiles (
```

```

    id          uuid          PRIMARY KEY REFERENCES
auth.users(id) ON DELETE CASCADE,
    username    text          UNIQUE NOT NULL,
    full_name    text          NOT NULL,
    avatar_url   text,
    bio          text,
    timezone     text          NOT NULL DEFAULT 'UTC',
    created_at   timestamptz NOT NULL DEFAULT now(),
    updated_at   timestamptz NOT NULL DEFAULT now()
);
CREATE TRIGGER profiles_updated_at BEFORE UPDATE
ON profiles
    FOR EACH ROW EXECUTE FUNCTION
handle_updated_at();

-- Auto-create profile row on signup
CREATE OR REPLACE FUNCTION handle_new_user()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY
DEFINER AS $$
BEGIN
    INSERT INTO profiles (id, full_name, username)
    VALUES (
        NEW.id,
        COALESCE(NEW.raw_user_meta_data ->> 'full_name',
'New User'),
        COALESCE(NEW.raw_user_meta_data ->> 'username',
split_part(NEW.email, '@', 1))
    );
    RETURN NEW;
END; $$;
CREATE TRIGGER on_auth_user_created AFTER INSERT
ON auth.users
    FOR EACH ROW EXECUTE FUNCTION
handle_new_user();

--
=====
=====
-- PROJECTS
--
=====
=====

```

```

CREATE TABLE projects (
  id      uuid      PRIMARY KEY DEFAULT
gen_random_uuid(),
  name    text      NOT NULL,
  description text,
  color   text,
  icon    text,
  is_archived boolean NOT NULL DEFAULT false,
  created_by uuid    NOT NULL REFERENCES
profiles(id),
  created_at timestampz NOT NULL DEFAULT now(),
  updated_at timestampz NOT NULL DEFAULT now()
);
CREATE TRIGGER projects_updated_at BEFORE UPDATE
ON projects
  FOR EACH ROW EXECUTE FUNCTION
handle_updated_at();

-- Auto-insert creator as owner
CREATE OR REPLACE FUNCTION handle_new_project()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO project_members (project_id, user_id,
role)
  VALUES (NEW.id, NEW.created_by, 'owner');
  RETURN NEW;
END; $$;
CREATE TRIGGER on_project_created AFTER INSERT ON
projects
  FOR EACH ROW EXECUTE FUNCTION
handle_new_project();

--
=====

=====

-- PROJECT MEMBERS
--
=====

=====

CREATE TABLE project_members (
  id      uuid      PRIMARY KEY DEFAULT
gen_random_uuid(),

```

```

    project_id uuid      NOT NULL REFERENCES
projects(id) ON DELETE CASCADE,
    user_id  uuid      NOT NULL REFERENCES profiles(id)
ON DELETE CASCADE,
    role     project_role NOT NULL DEFAULT 'member',
    joined_at timestamptz NOT NULL DEFAULT now(),
    UNIQUE(project_id, user_id)
);
CREATE INDEX ON project_members(project_id);
CREATE INDEX ON project_members(user_id);

```

```
--
```

```
-- INVITATIONS
```

```
--
```

```

CREATE TABLE invitations (
    id          uuid          PRIMARY KEY DEFAULT
gen_random_uuid(),
    project_id  uuid          NOT NULL REFERENCES
projects(id) ON DELETE CASCADE,
    invited_by  uuid          NOT NULL REFERENCES
profiles(id),
    invited_email text        NOT NULL,
    token       text          UNIQUE NOT NULL,
    role        project_role  NOT NULL DEFAULT
'member',
    status      invitation_status NOT NULL DEFAULT
'pending',
    expires_at  timestamptz   NOT NULL DEFAULT
(now() + interval '7 days'),
    created_at  timestamptz   NOT NULL DEFAULT
now()
);
CREATE INDEX ON invitations(project_id);
CREATE INDEX ON invitations(invited_email);

```

```
--
```

-- TASKS

--

```
CREATE TABLE tasks (  
  id      uuid      PRIMARY KEY DEFAULT  
gen_random_uuid(),  
  project_id uuid      NOT NULL REFERENCES  
projects(id) ON DELETE CASCADE,  
  title   text      NOT NULL,  
  description text,  
  status  task_status NOT NULL DEFAULT 'todo',  
  priority task_priority NOT NULL DEFAULT 'none',  
  assignee_id uuid      REFERENCES profiles(id) ON  
DELETE SET NULL,  
  created_by uuid      NOT NULL REFERENCES  
profiles(id),  
  due_date date,  
  position integer      NOT NULL DEFAULT 0,  
  created_at timestamptz NOT NULL DEFAULT now(),  
  updated_at timestamptz NOT NULL DEFAULT now()  
);  
CREATE INDEX ON tasks(project_id, status);  
CREATE INDEX ON tasks(assignee_id);  
CREATE TRIGGER tasks_updated_at BEFORE UPDATE  
ON tasks  
  FOR EACH ROW EXECUTE FUNCTION  
handle_updated_at();
```

--

-- MESSAGES

--

```
CREATE TABLE messages (  
  id      uuid      PRIMARY KEY DEFAULT  
gen_random_uuid(),  
  project_id uuid      NOT NULL REFERENCES  
projects(id) ON DELETE CASCADE,  
  sender_id uuid      NOT NULL REFERENCES
```

```

profiles(id),
  content  text    NOT NULL,
  mentions uuid[]  NOT NULL DEFAULT '{}',
  reply_to_id uuid  REFERENCES messages(id) ON
DELETE SET NULL,
  is_edited boolean NOT NULL DEFAULT false,
  is_deleted boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON messages(project_id, created_at);

--
=====

-- NOTIFICATIONS
--
=====

CREATE TABLE notifications (
  id      uuid    PRIMARY KEY DEFAULT
gen_random_uuid(),
  user_id uuid    NOT NULL REFERENCES profiles(id)
ON DELETE CASCADE,
  actor_id uuid    REFERENCES profiles(id),
  type    text    NOT NULL,
  title   text    NOT NULL,
  body    text,
  data    jsonb    NOT NULL DEFAULT '{}',
  is_read boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON notifications(user_id, is_read);

--
=====

-- ACTIVITY LOG
--
=====

CREATE TABLE activity_log (

```

```

id          uuid          PRIMARY KEY DEFAULT
gen_random_uuid(),
project_id  uuid          NOT NULL REFERENCES
projects(id) ON DELETE CASCADE,
actor_id    uuid          REFERENCES profiles(id),
action      text          NOT NULL,
entity_type text          NOT NULL,
entity_id   uuid,
metadata    jsonb         NOT NULL DEFAULT '{}',
created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON activity_log(project_id, created_at);

```

```
--
```

```

-- ENABLE REALTIME
-- Run in Dashboard → Database → Replication
-- ALTER PUBLICATION supabase_realtime ADD TABLE
messages;
-- ALTER PUBLICATION supabase_realtime ADD TABLE
notifications;

```

```
--
```

could you list functionalities or features that we can achieve from this database structure, not a query for that, but just feature/function like:

- * get project as per user has joined
- * task that assign to user
- * other accesses think from you end.

these list of functions/features are describe as much as possible so easy to understand and implement in mobile app.

You've got a **solid, production-ready schema** here — it already supports a full collaborative project management system (something like a lightweight mix of Jira + Slack). I'll break down **all possible features/functionality** you can build from it in a way that's easy to map into a mobile app.

User & Profile Features

These come from the `profiles` table + auth trigger.

Core

- User signup/login (handled via Supabase auth)
- Auto-create profile on signup
- View user profile
- Edit profile (name, bio, avatar, timezone)

Social / Identity

- Show user avatar & name across app
 - Mention users in chat (`mentions` array in messages)
 - Display user activity (via `activity_log`)
-

Project Features

From `projects` + `project_members`

Project Management

- Create new project
- Update project (name, description, icon, color)
- Archive/unarchive project
- Delete project (cascade deletes everything)

Project Access

- Get all projects user is part of
- Get projects created by user
- Role-based access:
 - Owner
 - Member

Membership

- Add/remove members
 - View all members of a project
 - Change member role (owner/member)
 - Auto-add creator as owner (already handled)
-



Invitation System

From `invitations`

Invite Flow

- Invite user via email
- Generate unique invite token
- Set role while inviting

Invitation Actions

- Accept invitation → add to `project_members`
- Decline invitation
- Expire invitation (after 7 days)
- Resend invitation

Views

- List pending invites for a project
 - List invitations sent by user
 - List invitations received (by email match)
-



Task Management (Core Feature)

From `tasks`

Task CRUD

- Create task
- Update task
- Delete task
- View task details

Task Organization

- Filter by:
 - status (`todo` , `in_progress` , etc.)
 - priority (`low` , `high` , etc.)
 - assignee
- Sort by:
 - due date
 - priority
 - position (for drag-drop UI)

Assignment

- Assign task to user
- Unassign task
- Get tasks assigned to current user

Status Workflow

- Move task across stages:
 - backlog → todo → in_progress → done
- Kanban board UI support

Priority Handling

- Highlight urgent tasks
- Filter high-priority tasks

Deadlines

- Set due date
 - Show overdue tasks
 - Upcoming tasks view
-



Messaging / Chat (Realtime Feature)

From `messages`

Chat Features

- Send message in project
- Edit message
- Delete message (soft delete)
- Reply to message (threading)

Mentions

- Mention users → trigger notifications
- Highlight mentions in UI

Chat UI

- Group chat per project
 - Threaded replies
 - Show message history (sorted by `created_at`)
-



Notifications System

From `notifications`

Notification Types (you can define)

- Task assigned
- Mention in message
- Invitation received
- Task status changed
- Project updates

Features

- Get unread notifications
- Mark as read
- Real-time notifications (Supabase realtime)

Personalization

- Notifications per user
 - Store extra data in `jsonb`
-



Activity Log (Audit Trail)

From `activity_log`

Tracking

- Track actions like:
 - Task created/updated/deleted
 - Member added/removed
 - Message sent
 - Status changes

Features

- Show project activity timeline
 - Filter by:
 - user
 - entity (task/project)
 - Debugging & audit history
-

Realtime Features (Supabase)

From enabled tables

Live Updates

- Real-time chat updates
 - Live notifications
 - Task updates instantly reflect on all devices
-



Advanced Features You Can Build

These are **derived features** using your schema.

Dashboard

- Total tasks per project
- Tasks per status
- Tasks per user
- Overdue tasks count

Productivity

- User workload (tasks assigned)
- Task completion rate
- Activity frequency

Search

- Search tasks by title/description
 - Search messages
 - Search users in project
-



Access Control (Important)

Derived from `project_members`

Permissions

- Only members can:
 - View project
 - View tasks/messages
- Only owners can:
 - Delete project

- Manage roles
-



Mobile App Feature Mapping (Very Practical)

Here's how it translates into screens:

Auth Screens

- Login / Signup

Home

- List of user projects

Project Screen

- Tabs:
 - Tasks (Kanban)
 - Chat
 - Members
 - Activity

Task Screen

- Task list
- Task details
- Create/edit task

Chat Screen

- Real-time messaging
- Mentions

Notification Screen

- List of notifications
- Mark as read

Profile Screen

- User details
 - Edit profile
-



Bonus Features You Can Add Later

Your schema already supports:

- File attachments (add column/table)
 - Comments on tasks (reuse messages or new table)
 - Subtasks (self-referencing tasks)
 - Labels/tags (new table)
 - Time tracking
 - Push notifications
-

Key Strength of Your Schema

- Clean separation of concerns
 - Scalable (multi-project, multi-user)
 - Realtime-ready
 - Role-based access already baked in
-

If you want, I can next:

- Design **API endpoints** (REST or Supabase RPC)
- Suggest **mobile UI structure** (Flutter/React Native)
- Or create **feature-wise backend service layer** (C# / Node)